



Mars Rover Embedded Integration

Alaa Qutishat 20231035

Tania Zeyad Fostuq 20230343

Omar Osama Abu Mohammad 20230479

Supervisor:
Dr.Heba abdelnabi



Table of Contents

1. System Design Description & Logic Flow 2

2. Port Addresses and Peripheral Connections 4

3. Keypad Scanning and Mapping Table 4

4. Keypad Debouncing Solution 4

5. 7-Segment Display Interfacing and Truth Table 5

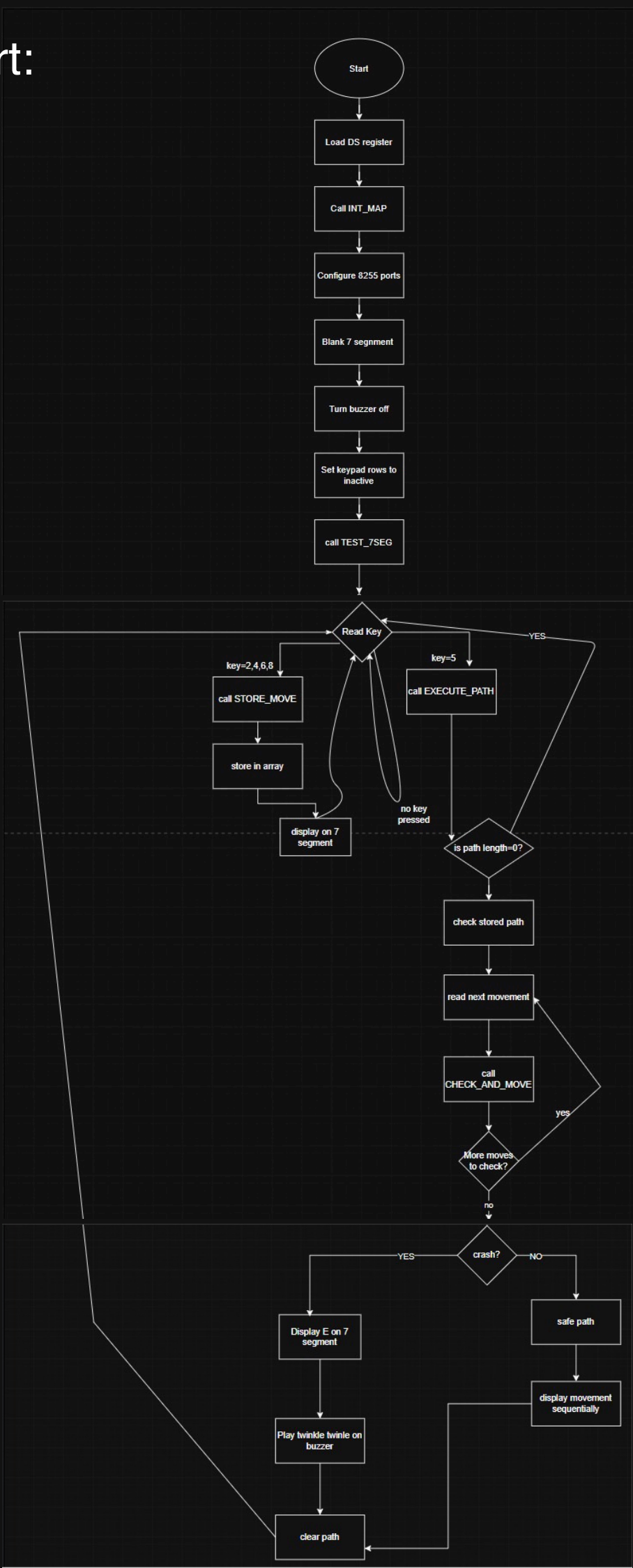
6. Screenshots of Final Results 6

1. System Design Description & Logic Flow

This project transitions the software-based Mars Rover simulation into a fully functional embedded system. The hardware body utilizes the Intel 8086 microprocessor, an 8255 PPI, a Matrix Keypad, and a 7-Segment Display. The system operates as an intelligent embedded unit with the following logical flow:

- **Initialization & Memory Mapping:** The 8086 configures the 8255 Control Word (88H), clears the 7-segment display, turns off the buzzer, and runs a startup display test (`TEST_7SEG`). It then generates the 8x8 terrain grid directly into the memory map.
- **Mission Input Phase:** The system continuously scans the physical Matrix Keypad. When the user presses movement keys (2, 4, 6, 8), the microprocessor briefly flashes the corresponding character on the display to acknowledge the input, but does not execute the move immediately. Instead, it stores the sequence in a dedicated array in memory.
- **Execution Trigger:** Pressing the 5 key (EXECUTE MISSION) exits the input phase and initiates path calculation. If the path length is zero, it loops back to read keys.
- **Path Validation & Sequential Execution:**
 - **Phase 1 (Checking):** The microprocessor first loops through the stored sequence and simulates the movement on the internal memory grid (`CHECK_AND_MOVE`) to verify the entire path.
 - **Phase 2 (Safe Execution):** If the entire path is safe (No Crash), it outputs the stored moves to the 7-Segment Display sequentially. A mandatory software delay ensures the display holds each character for approximately 1 second.
 - **Critical Failure Handling:** If the calculated path hits an obstacle or a boundary (Crash = YES), the system aborts immediately. It displays 'E' (Error) on the 7-Segment Display and plays the "Twinkle Twinkle" melody on the speaker.
- **Reset (Clear Path):** Regardless of whether the mission was successful or resulted in a crash, the system ultimately clears the stored path memory, resetting the length to zero, and loops back to the input phase to await a new mission.

Flowchart:



2. Port Addresses and Peripheral Connections

The address decoding circuit utilizes the 74LS138 decoder. Utilizing the specific project constraint for our group, the Base Address for Port A is calculated using the last two digits of the university ID (35H). The address is the result of 35H ANDing with F8H: 35H AND F8H = 30H.

The 8255 operates in Mode 0. The calculated addresses and connections are:

- **Port A (Address 30H - Configured as OUTPUT):** Connected to the Common Anode 7-Segment Display to output telemetry ('F', 'b', 'L', 'r', 'E').
- **Port B (Address 32H - Configured as OUTPUT):** Connected to an NPN Transistor (2N2222) driving the Sounder/Speaker circuit for the custom critical failure alarm melody.
- **Port C (Address 34H):** Interfaced with the 3x4 Matrix Keypad. The Lower Nibble (PC0-PC3) is configured as OUTPUT to drive keypad rows, and the Upper Nibble (PC4-PC6) is configured as INPUT to read keypad columns.
- **Control Register (Address 36H):** Receives the control word 88H to initialize the ports.

3. Keypad Scanning and Mapping Table

- The keyboard scanning process uses an active-low technique. The microprocessor drives one row LOW at a time and checks the columns for a LOW signal to detect a pressed key.

Key Pressed	Row Activated (Output)	Column Detected (Input)	Command Mapped
2	Row 1 (PC0)	Col 2 (PC5)	Up
4	Row 2 (PC1)	Col 1 (PC4)	Left
5	Row 2 (PC1)	Col 2 (PC5)	EXECUTE MISSION
6	Row 2 (PC1)	Col 3 (PC6)	Right
8	Row 3 (PC2)	Col 2 (PC5)	Down

4. Keypad Debouncing Solution

Mechanical switches generate electrical noise (bouncing) when pressed or released. We solved the debouncing problem of the keypad using a robust Software Delay Method.

When a row is driven LOW, a SETTLE subroutine (short loop of NOP instructions) is called to allow voltages to stabilize before reading the columns. Once a valid key is registered, a WAIT_KEY_RELEASE procedure is invoked. This procedure polls the keypad until all keys are released (verifying no buttons are held down), followed by an additional DELAY_200MS subroutine before the main input loop restarts. This entirely prevents false multi-press registrations and accidental sequence duplications

5. 7-Segment Display Interfacing and Truth Table

The 7-segment display is directly interfaced with the 8255 via Port A. We used a Common Anode configuration, meaning a logic 0 turns a segment ON. Data pins D0-D7 of Port A correspond to segments a-g and the decimal point (dp).

Logic Truth Table (0 = ON, 1 = OFF):

Character	dp (PA7)	g (PA6)	f (PA5)	e (PA4)	d (PA3)	c (PA2)	b (PA1)	a (PA0)
F	1	0	0	0	1	1	1	0
b	1	0	0	0	0	0	1	1
L	1	1	0	0	0	1	1	1
r	1	0	1	0	1	1	1	1
E	1	0	0	0	0	1	1	0

Figure 1: Full Proteus schematic

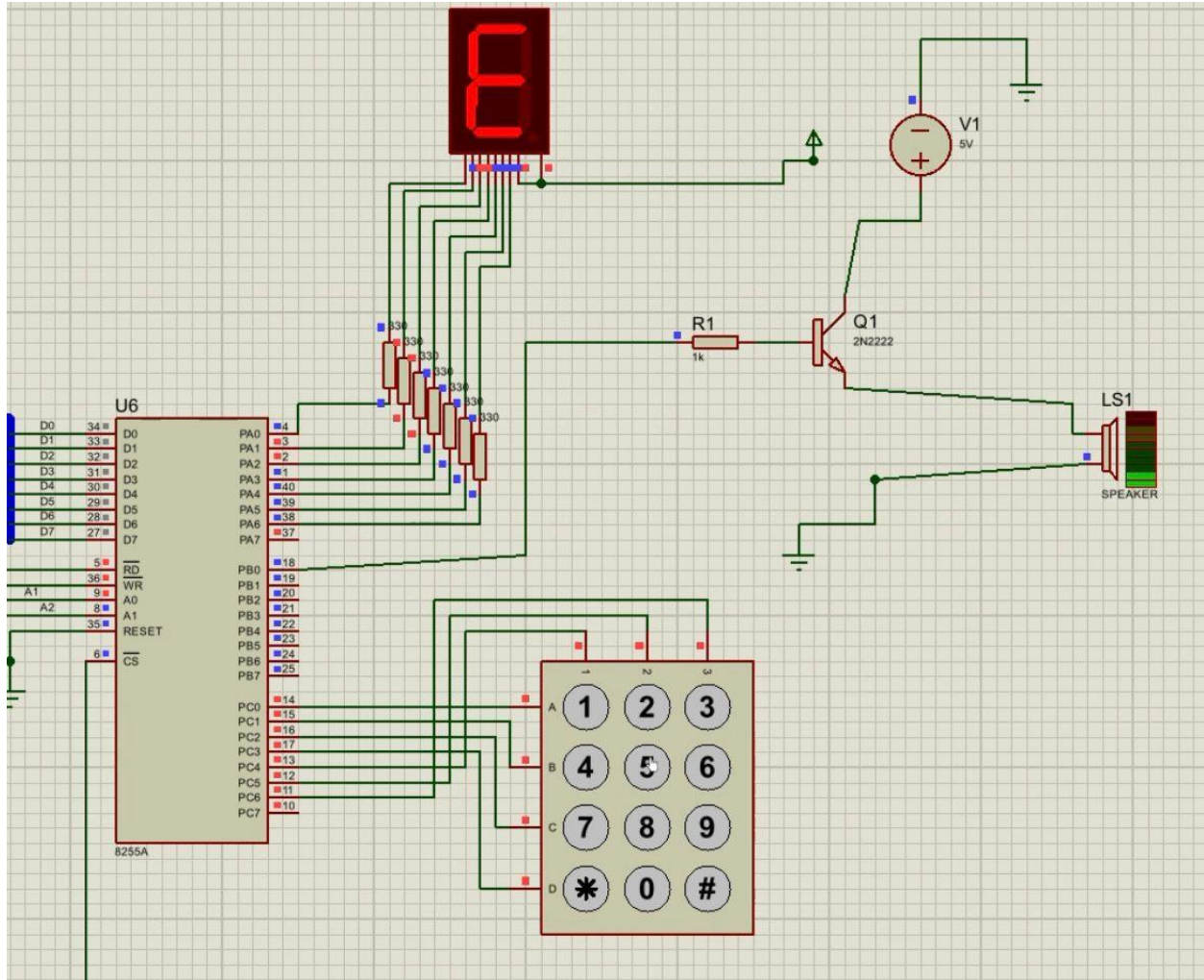


Figure 2: The system in Critical Failure mode. The 7-segment display outputs 'E' (Error), while Port B actively drives the transistor to play the synthesized alarm melody.

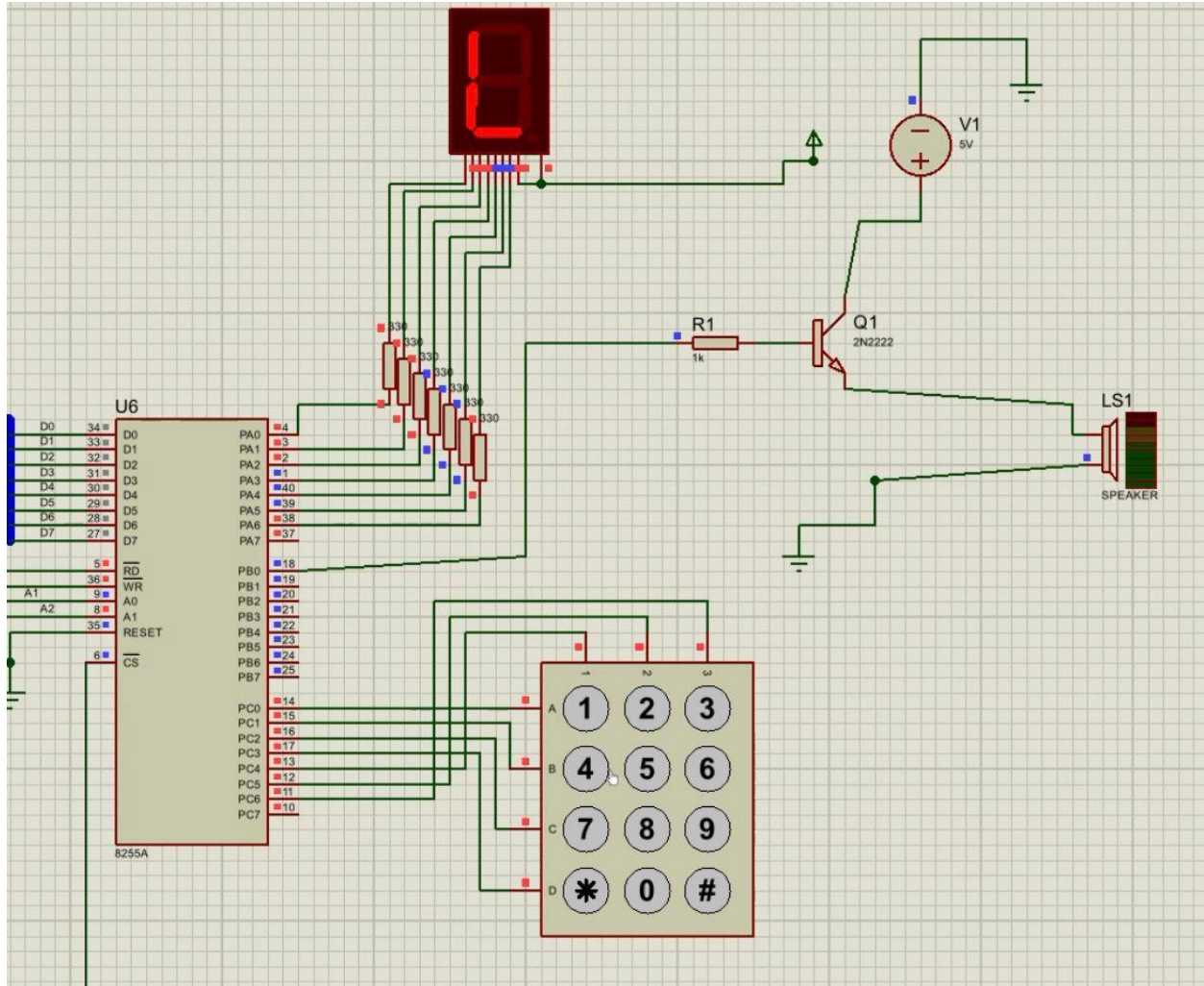


Figure 3: The 7-segment display outputting the character 'L', indicating a 'Left' movement during the sequential execution of a safe path.

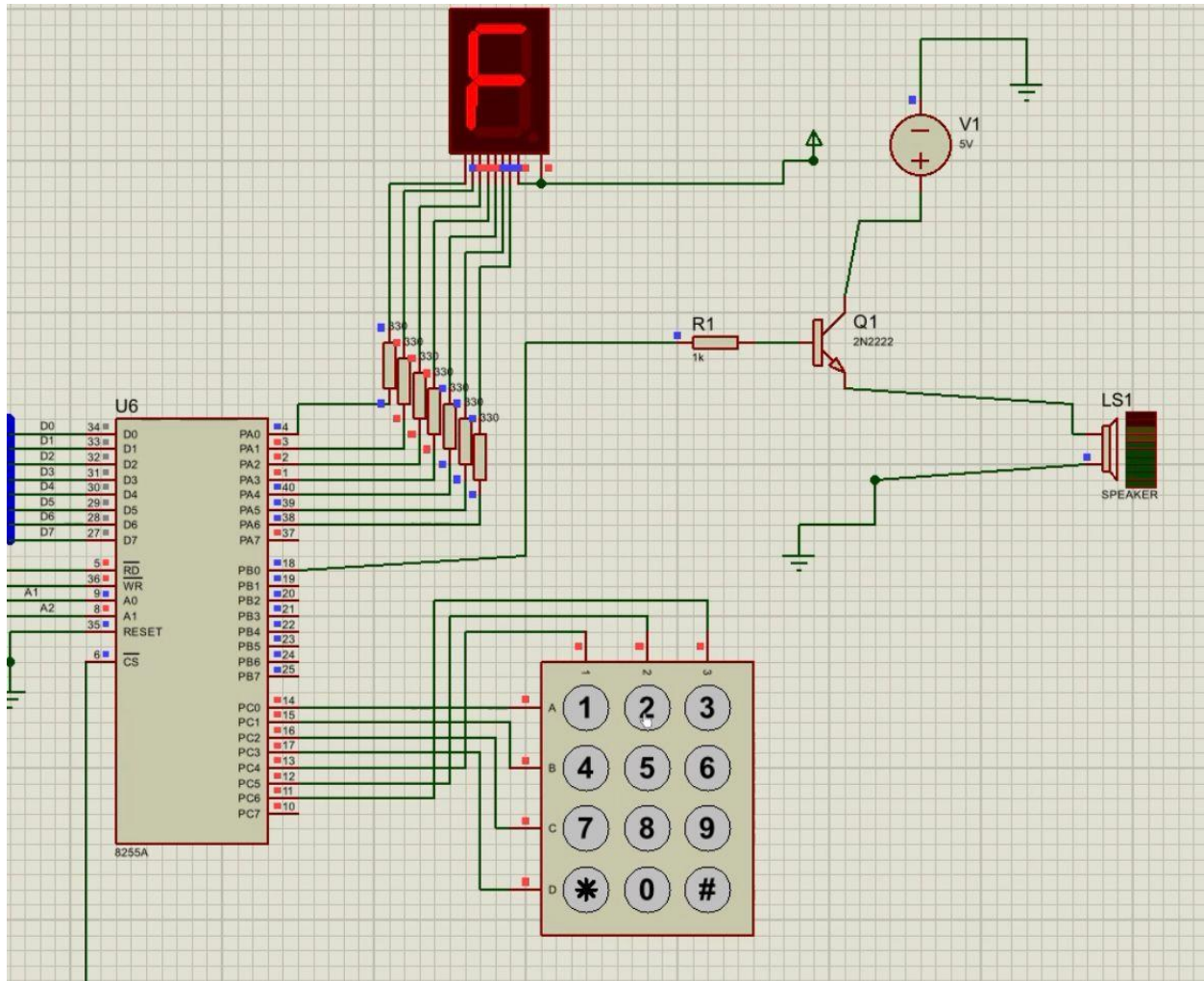


Figure 4: The 7-segment display outputting the character 'F', indicating a 'Forward' or 'Up' movement during the sequential execution of a safe path.

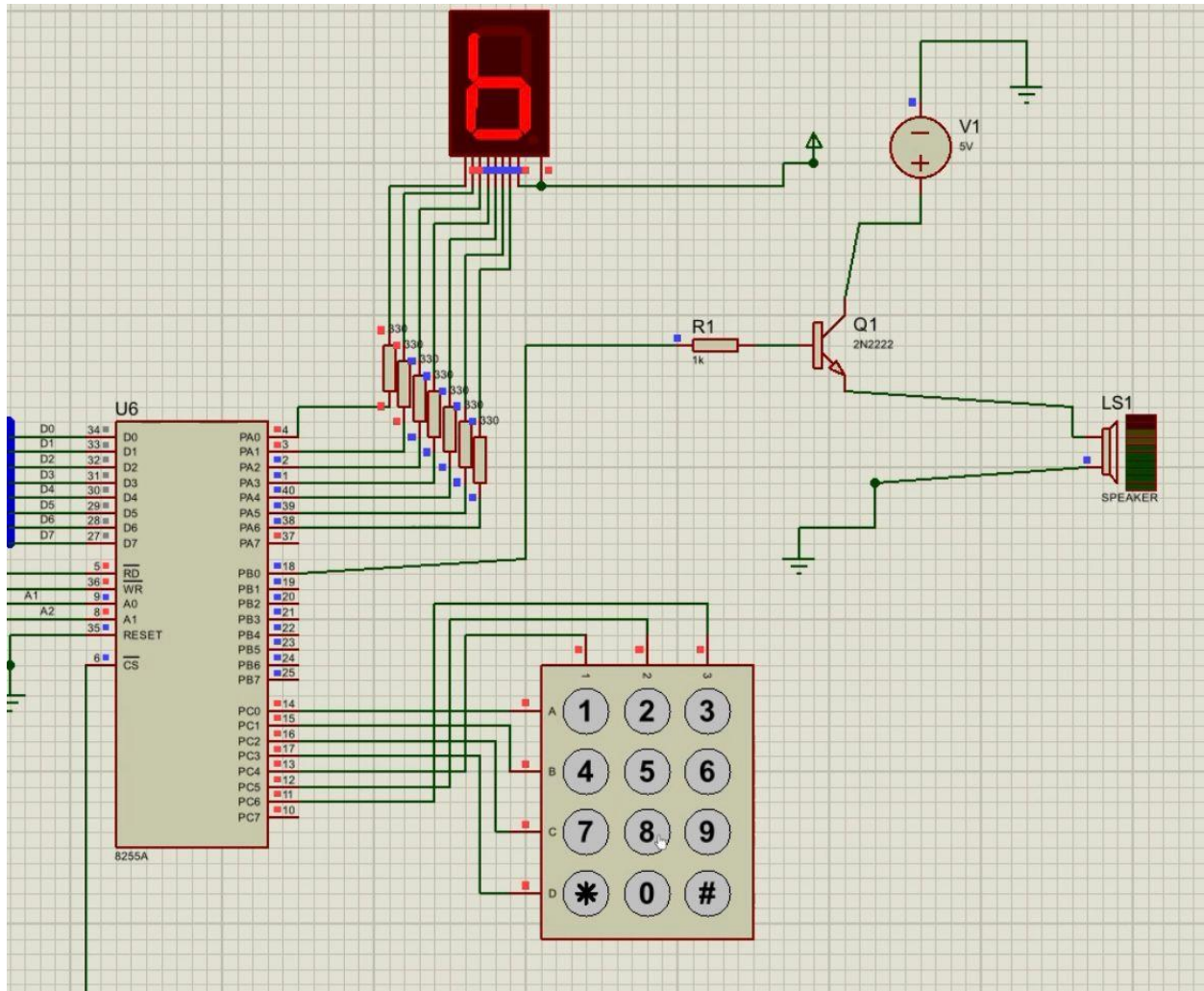


Figure 5: The 7-segment display outputting the character 'b', indicating a 'Backward' or 'Down' movement during the sequential execution of a safe path.

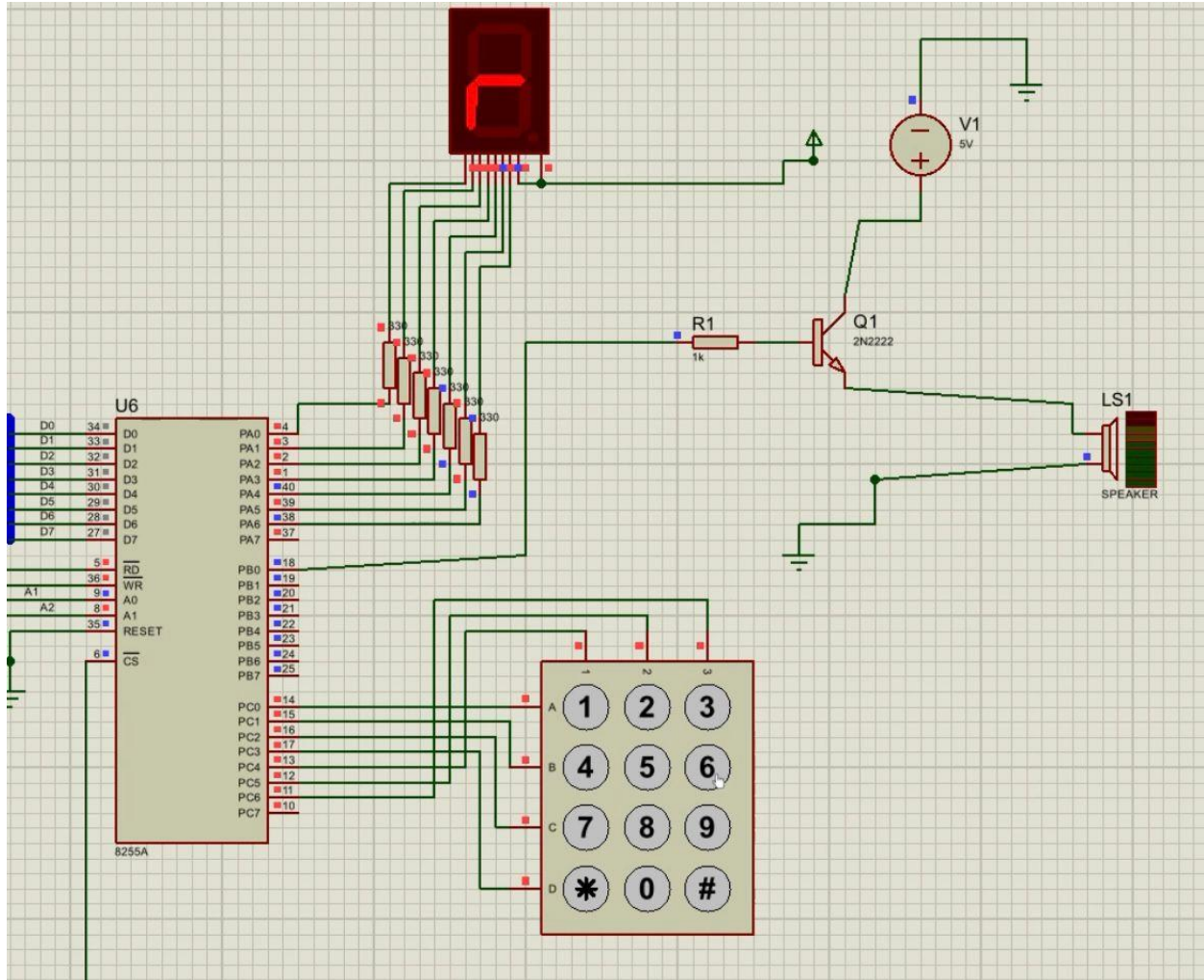


Figure 6: The 7-segment display outputting the character 'r', indicating a 'Right' movement during the sequential execution of a safe path.